

# Ciclos y Recursión

---

Robin Omar Buezo Díaz



Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas

# Indice

## Contenido

|                                                     |   |
|-----------------------------------------------------|---|
| Indice .....                                        | 1 |
| Competencia(s) .....                                | 2 |
| Definición de ciclos recursivos .....               | 2 |
| Objetivo.....                                       | 2 |
| Introducción.....                                   | 2 |
| Contenido .....                                     | 2 |
| Pongamos en Práctica la Teoría .....                | 2 |
| Caso de Estudio 1.....                              | 2 |
| Caso de Estudio 2.....                              | 3 |
| Conclusiones .....                                  | 3 |
| Resolución de problemas mediante recursividad ..... | 3 |
| Objetivo.....                                       | 3 |
| Introducción.....                                   | 3 |
| Contenido .....                                     | 3 |
| Pongamos en Práctica la Teoría .....                | 4 |
| Caso de Estudio 1.....                              | 4 |
| Caso de Estudio 2.....                              | 4 |
| Conclusión.....                                     | 4 |
| Conclusión General .....                            | 4 |
| Referencias .....                                   | 4 |

## Competencia(s)

Desarrolla hechos, reglas, expresiones y predicados recursivos mediante el uso de cláusulas, ciclos, listas, unificación y cortes en Tau Prolog para modelar bases de conocimiento y resolver problemas de inferencia lógica.

## Definición de ciclos recursivos

### Objetivo

Comprender cómo se representan los ciclos a través de recursión en programación lógica.

### Introducción

Prolog no utiliza estructuras de bucle como otros lenguajes, sino que simula los ciclos mediante recursión. Una regla se llama a sí misma hasta cumplir una condición de salida. Este mecanismo se conoce como recursión y permite repetir tareas de forma controlada.

### Contenido

Un ciclo recursivo se compone de dos partes:

1. **Caso base**, que detiene la recursión.
2. **Caso recursivo**, que llama a la misma regla con nuevos argumentos.

Ejemplo clásico: contar elementos de una lista.

```
prolog
CopiarEditar
contar([], 0).
contar([_ | T], N) :- contar(T, R), N is R + 1.
```

Cada llamada elimina un elemento de la lista hasta llegar a una vacía. En ese momento, Prolog retorna el resultado acumulado.

## Pongamos en Práctica la Teoría

### Caso de Estudio 1

En un sistema de control de productos, se necesita saber cuántos artículos hay registrados. La base contiene registros como producto(laptop), producto(mouse), producto(teclado). Se crea una lista y se cuenta usando un predicado recursivo.

## Caso de Estudio 2

Para procesar las edades de un grupo de personas, se puede aplicar un predicado que sume todas las edades de una lista, usando una estructura similar a la anterior.

## Conclusiones

Los ciclos recursivos permiten resolver tareas repetitivas sin necesidad de estructuras externas. En Prolog, la recursión es esencial para operar sobre listas y estructuras anidadas.

# Resolución de problemas mediante recursividad

## Objetivo

Aplicar técnicas recursivas para resolver problemas prácticos como búsqueda, acumulación de datos y transformaciones.

## Introducción

La recursión no solo sirve para contar, sino también para resolver problemas más elaborados. Al diseñar la lógica de un problema, se puede descomponer en partes que se resuelven recursivamente. Esto hace posible implementar funciones como ordenamiento, validación o generación de combinaciones.

## Contenido

Algunos problemas comunes que se resuelven por recursividad son:

- Sumar los elementos de una lista
- Verificar si un elemento está contenido en una lista
- Invertir una lista
- Transformar datos

Ejemplo: determinar si un número está en una lista.

```
prolog
CopiarEditar
pertenece(X, [X | _]).
pertenece(X, [_ | T]) :- pertenece(X, T).
```

Prolog recorrerá cada elemento hasta encontrar coincidencia o llegar al final.

## Pongamos en Práctica la Teoría

### Caso de Estudio 1

Una aplicación de tutoría quiere saber si un estudiante está registrado en una lista. Se aplica el predicado `pertenece(Nombre, Lista)` para comprobarlo. Este proceso no necesita bucle externo, solo recursión.

### Caso de Estudio 2

En un editor de textos se desea contar cuántas veces aparece una palabra en una lista de palabras. Se crea un predicado recursivo que evalúe si el primer elemento es igual al buscado y sume uno si es el caso.

## Conclusión

La recursividad es una herramienta clave para resolver problemas complejos sin estructuras iterativas. Permite mantener un diseño limpio y lógico, adaptado al paradigma declarativo de Prolog.

## Conclusión General

El uso de ciclos recursivos en Prolog representa una forma eficiente y natural de trabajar con datos secuenciales. Comprender esta técnica es fundamental para todo desarrollo lógico, desde simples conteos hasta algoritmos más avanzados.

## Referencias

Bratko, I. (2012). *Prolog Programming for Artificial Intelligence*. Pearson  
Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog*. Springer  
Sterling, L., & Shapiro, E. (1994). *The Art of Prolog*. MIT Press

